

≡ Hide menu

Introduction to Vector Fields

Painting a Vector Field

Neighboring Vectors and Particles

Video: Neighboring Vectors17 min

Video: Neighboring Vectors 29 min

Reading: Text: Neighboring vectors10 min

Video: Calibrating the Paint Tool10 min

Video: Particles Over Vector Field17 min

Practice Assignment: Practice Quiz: Vector Field Interactions30 min

Graded Assignment: Week 2 Quiz: Vector Fields30 min

Ungraded App Item: Gallery: Week 2 Reflections1h

Text: Neighboring vectors

In order to apply changes not only to one vector but to many vectors in the vector field, we will learn a few new approaches to accessing neighbors in a grid.

In this first example, we will explicitly write a list with the neighbors we are looking for:

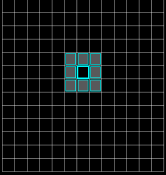
NEIGHBORING VECTORS

#Example

moves = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 0), (0, 1), (1, -1), (1, 0), (1, 1)]

for dx, dy in moves:

#calculate the index from position



1 layers of neighbors

This is how this would look in the code:

```
1 def get_adjacent_vectors(self, x, y):
2     # List to store the indices of adjacent vectors
3     adjacent_vectors = []
4
5     # Define relative moves to get adjacent positions
6     # Each tuple represents a direction (dx, dy)
7     moves = [ (-1, -1), (-1, 0), (-1, 1),
8               (0, -1),           (0, 1),
9               (1, -1), (1, 0), (1, 1) ]
10
11    # Iterate through each possible adjacent direction
12    for dx, dy in moves:
13        # Calculate the new x and y positions by adding the relative moves
14        n_x = x + (dx * self.cell_size_x)
15        n_y = y + (dy * self.cell_size_y)
16
17        # Convert these positions to column and row indices in the vector field grid
18        n_x = int(n_x / (float(self.world_x) / self.cols))
19        n_y = int(n_y / (float(self.world_y) / self.rows))
20
21        # Calculate the flat list index of the adjacent vector
22        index = n_y * self.cols + n_x
23
24        # Check if the calculated position is within the bounds of the vector field
25        if 0 <= n_x < self.cols and 0 <= n_y < self.rows:
26            # Add the index to the list of adjacent vectors
27            adjacent_vectors.append(index)
28
29    # Return the list of indices of adjacent vectors
30    return adjacent_vectors
31
```

The function calculates the indices of all vectors adjacent to a given position (x, y) in the vector field.

- **Adjacent Positions:** It defines a set of relative moves to find adjacent positions. These moves represent the eight directions surrounding a given point in a 2D grid.
- **Index Calculation:** For each adjacent direction, the function calculates the new position and then converts it to the corresponding row and column in the grid. From this, it calculates the index in the flat list representation of the vector field.
- **Bounds Checking:** The function ensures that each adjacent position is within the bounds of the vector field before adding its index to the list.
- **Return Value:** It returns a list of indices corresponding to the adjacent vectors.

Now, let's learn how to automate the construction of the neighbor list, allowing us to dynamically define how many rows of neighbors we would like to access.

NEIGHBORING VECTORS

#Example

moves = []

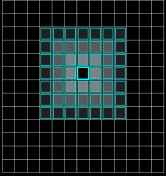
for i in range(-layers, layers + 1):

for j in range(-layers, layers + 1):

if i != 0 or j != 0:

moves.append((i, j))

return moves



3 layers of neighbors

```
15
```

The function is designed to create a list of coordinates representing relative positions or **moves** in a grid, centered around a specific point.

- **Layers:** The parameter layers determine how far from the center point (0, 0) the moves should be generated. A larger number of layers means a wider area around the center will be covered.
- **Generating Moves:** The function iterates through a range of values from -layers to layers inclusive for both the x (i) and y (j) coordinates. This creates a square grid of coordinates.
- **Excluding the Center:** The condition if i != 0 or j != 0 ensures that the central point (0, 0) is not included in the list of moves. This is important if the center is meant to be the reference or starting point and should not be considered a 'move'.
- **Result:** The function returns a list of tuples, each representing a move or offset from the center. For example, if layers is 1, the function will return offsets representing all immediate adjacent positions (up, down, left, right, and the four diagonals).

This code can now replace the manually written calculations of moves defining the adjacent vectors.

```
1 ...
2 moves = [ (-1, -1), (-1, 0), (-1, 1),
3           (0, -1),           (0, 1),
4           (1, -1), (1, 0), (1, 1)
5         ]
6 ...
7 moves = self.generate_moves(num_layers)
```

Go to next item

✓ Completed

👍 Like

👎 Dislike

📄 Report an issue